

The Optimal Expert Prefetch Horizon Is Set by Precision

ANONYMOUS AUTHOR(S)

The optimal prefetch horizon for expert weights is set by precision: $\Phi = B/P_{lo}$, the device budget divided by a low-precision copy of every expert, decides whether those bytes buy residency or lookahead. Precision assignment assumes that low-precision copy always fits, while prefetching assumes expert size is fixed and a longer horizon always helps; the two assumptions exchange places at $\Phi = 1$. When $\Phi \geq 1$ the minimizing horizon is zero and leftover bytes raise the precision of experts the router uses; when $\Phi < 1$ it is the smallest lookahead that hides the predicted misses inside the overlap window and the budget, and prefill versus decode crosses that boundary inside one request. DynaByte computes this horizon from measured expert sizes, tracks it by at most one layer per period, and changes precision only while the byte budget and the latency band both hold. A sweep of budgets, models, and devices tests the predictions against precision-only residency, horizon-only prefetch, and a composition that lets each policy spend the full budget.

CCS Concepts: • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → **Memory management**; • **Computer systems organization** → *Architectures*.

Additional Key Words and Phrases: mixture of experts, memory management, prefetching, quantization, performance modeling

ACM Reference Format:

Anonymous Author(s). 2027. The Optimal Expert Prefetch Horizon Is Set by Precision. *Proc. ACM Meas. Anal. Comput. Syst.* 1, 1 (September 2027), 20 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Draft status. This manuscript follows the SIGMETRICS 2027 acmsmall rules (at most 20 single-column pages of technical content, unlimited references, double-anonymous review). Tables that depend on the joint re-measurement are marked in red and left blank. Preliminary routing statistics in Section 2.2 are measurements already in hand; they are not end-to-end results for DynaByte.

1 Introduction

A Mixture-of-Experts (MoE) layer routes each token to a few expert subnetworks, so parameter count grows without a proportional increase in per-token arithmetic [14, 23, 31]. Deployed models such as Mixtral, DeepSeek-V2, and Qwen3 keep tens to hundreds of billions of parameters resident in the expert pool while activating only a few billion per token [10, 19, 36]. On one accelerator the expert pool dominates the footprint: in the models we study it is about 95–96% of the weights. Device memory must also hold non-expert parameters, activations, the KV cache, and runtime buffers. The resulting constraint is a byte budget B for expert weights. Every expert is in one of three states under that budget: absent from the device, resident at low precision, or resident at high precision. The same bytes also determine how many future layers of experts can be prefetched before the cache overflows.

Two lines of work each freeze one of those choices. Precision-oriented systems compress experts with post-training quantization and, in the adaptive case, spend a scarce high-precision quota on experts that the router uses heavily [12, 15, 25, 35]. Their operating assumption is that a low-precision copy of every routed expert stays addressable on the device, so a cache miss never occurs. Prefetch-oriented systems treat device memory as a cache and overlap expert transfers with compute by fetching the experts of the next S layers [13, 32, 34, 40]. Their operating assumption is that expert byte size is fixed, so the only decision is which experts to stage and how large S should

be. Each assumption defines a real deployment, and each is the wrong deployment for the other budget.

The optimal expert prefetch horizon is set by precision, through one ratio. Let P_{lo} be the number of bytes required to store every routed expert at low precision, and P_{hi} the same quantity at high precision.

$$\Phi = B/P_{lo}.$$

Section 2.3 derives three regimes from Φ and from the overlap identity that a horizon of S layers hides at most $S \cdot T_\ell \cdot C$ bytes of transfer, where T_ℓ is the per-layer compute time and C is the effective host-to-device bandwidth. When $\Phi \geq P_{hi}/P_{lo}$, the pool fits at high precision and neither actuator is needed. When $1 \leq \Phi < P_{hi}/P_{lo}$, the pool fits at low precision and does not fit at high precision. The latency-minimizing horizon is then $S^* = 0$: every expert can stay resident, and leftover bytes, $B - P_{lo}$, buy high-precision pins. When $\Phi < 1$, some experts are absent. S^* is the smallest horizon that hides the predicted miss set and whose speculative bytes still fit in B ; high-precision pins are taken only from the slack above that horizon. A second, faster variable sits inside the third regime. Prefill and large batches touch a large fraction of the experts in a layer, while the next decode step touches far fewer (Section 2.2). The feasible set for S can be empty during prefill and non-empty during decode at the same Φ . The active actuator therefore changes inside one request.

DynaByte is a runtime that tracks this model. At startup it measures packed expert sizes and classifies Φ . A slow loop maintains high-precision identities with an exponentially smoothed hotness score and a hysteresis margin, and it changes the pin count by at most one expert per layer per period. A fast loop moves S by at most one layer per period toward the model's S^* , using the precision at which the expert will actually be transferred. Admission control reserves bytes before any promotion, demotion, or prefetch, and a versioned handle becomes visible to the forward pass only after its copy completes. Routing semantics are unchanged: the runtime does not drop experts and does not alter top- k .

This paper makes four contributions.

- We formulate expert-weight memory as one byte budget with three residency states, and we show that the ratio Φ plus the overlap identity predict which actuator—high-precision pins, a low-precision resident floor, or a prefetch horizon—can reduce exposed transfer latency.
- We give a two-timescale controller that tracks the predicted horizon with a bounded update and spends residual bytes on high-precision pins only when the latency band remains feasible.
- We specify the memory invariant and the publication protocol required for precision migration and prefetch to share that budget without a forward-path dependence on an in-flight copy.
- We specify a measurement protocol that sweeps Φ on one stack, tests the model's predictions before any system comparison, and treats precision-only residency, horizon-only prefetch, and their naive composition as the baselines that occupy the boundary policies.

On acceptance we will release the controller, the budget-accounting runtime, the routing-trace tools, and the configuration files for every reported point, hosted in a long-lived archive. Section 2 states the serving model and the preliminary measurements. Section 2.3 defines Φ and the predictions. Section 3 describes the controller. Section 4 records the implementation. Section 5 states the hypotheses and the protocol. Section 6 records limitations. Section 7 places the boundary policies in prior work. Section 8 concludes.

2 Background and Motivation

2.1 Expert Bytes on One Accelerator

An MoE layer ℓ holds E_ℓ routed experts and sends each token to k_ℓ of them. Expert weights are the movable object. Non-expert parameters, the KV cache, activations, and a conversion workspace are sized for a declared maximum batch and sequence length and are reserved outside B . The runtime's allocation problem is only the expert budget. Host memory holds a packed low-precision copy and a packed high-precision copy of each expert. The device holds at most one live copy. A live copy occupies either m_ℓ^{lo} or m_ℓ^{hi} bytes, both measured from the packed representation rather than from a nominal bit width. Promoting an expert therefore costs $m_\ell^{\text{hi}} - m_\ell^{\text{lo}}$ extra device bytes if a low-precision copy was already resident, and it costs m_ℓ^{hi} bytes if the expert was absent.

Three published mechanisms cover different corners of this state space. Uniform post-training quantization assigns every expert the same bit width offline [11, 15, 25, 39]. The configuration is feasible only when the chosen width's pool fits in B , and it cannot move fidelity when the router's hot set changes. Cross-layer prefetch issues transfers for the experts predicted over the next S layers on a side stream, overlapping them with the compute of layers $\ell, \dots, \ell+S-1$ [13, 18, 34]. S does not change routing. It changes only how many distinct expert bytes the cache must absorb. Mixed-precision offload uses a cheaper representation on the miss path so that a cache fill moves fewer bytes [17, 35]. The bit width in that design is a property of the transfer, not a pin that is protected from eviction.

2.2 Preliminary Measurements

The joint protocol in Section 5 repeats the following measurements on one runtime. We report them here because they fix the quantities the model has to explain. All three were taken on an NVIDIA RTX A6000 [27]. They are not comparable to each other as end-to-end system results: the density and hot-set traces use the model's native routing with experts forced to the low tier, and the horizon sweep uses a separate 20 GB residency cap.

Activation density swings inside one request. We define the unique-expert ratio of a step as the number of distinct experts selected by top- k routing, divided by E_ℓ , averaged over routed layers. The average is over five fixed prompt blocks. Batch b uses the first b prompts of a 32-prompt block. Prefill is the unique set over all non-padding tokens of a prompt capped at 2,048 tokens. Decode is the unique set of the single next token, not an accumulation over a long generation. Table 1 shows the ratio on three models. On Qwen3-30B at batch 32 the prefill ratio is 94.0% and the next decode step is 26.0%. On Phi-3.5-MoE the prefill ratio is essentially the entire pool at every batch we measured. A horizon chosen for the decode working set does not describe the prefill working set of the same request.

The hot set is a property of the task. On Qwen3-30B, Layer 15, we counted every selected top- k dispatch under three full workloads (WikiText, GSM8K, and HumanEval) with the scheduler frozen and every expert held at low precision. The ten most frequently selected experts were disjoint across the three workloads. A high-precision assignment computed on one calibration corpus therefore pins the wrong experts on the other two tasks. The slow loop has to be allowed to change identities. It also has to be slow: a swap replaces a resident block and consumes migration bandwidth.

The horizon has a U-shaped latency curve when the pool does not fit. Under a 20 GB device cap that forces expert absence, latency against S on three smaller MoE models was U-shaped. Below the minimum, transfers do not finish inside the compute of the next S layers and the critical path accumulates exposed wait. Above the minimum, the speculative set exceeds the cap, evictions

Table 1. Preliminary unique-expert activation ratio (%), RTX A6000. Each cell averages routed layers and five prompt blocks. Prefill accumulates the prompt; decode is one subsequent token. The joint protocol repeats this table before any system comparison.

Model	Stage	1	2	4	8	16	32
Qwen3-30B	Decode	6.3	7.9	11.8	16.1	20.8	26.0
	Prefill	59.8	70.0	82.9	89.7	92.4	94.0
Qwen3-Next-80B	Decode	1.9	3.7	6.7	14.9	25.4	39.2
	Prefill	24.6	35.3	48.8	67.1	78.9	86.2
Phi-3.5-MoE	Decode	12.5	22.7	37.1	59.3	77.8	88.8
	Prefill	98.2	98.8	99.4	99.8	99.9	100.0

Phase III $\Phi < 1$ absent experts	Phase II $1 \leq \Phi < \rho$ all resident at low	Phase I $\Phi \geq \rho$ all resident at high
tighter budget		looser budget

Fig. 1. Regimes of the expert-byte budget. $\rho = P_{hi}/P_{lo}$. Phase II spends leftover bytes on high-precision pins and sets the horizon to zero. Phase III spends bytes first on the smallest feasible horizon, then on pins. The figure is the model's partition; Section 5 checks it against measured latency.

increase, and later misses reload experts that a smaller horizon would have kept. The location of the minimum differed by model on the same device. Holding S fixed and increasing the batch from 1 to 32 increased the waiting component by up to $2.5\times$. At fixed batch size, requests with higher intra-batch embedding dispersion activated more distinct experts and showed up to 300% end-to-end latency variation against low-dispersion requests of the same length. Effective host-to-device bandwidth across the devices used in that sweep spans roughly an order of magnitude, from a low-bandwidth discrete GPU to NVIDIA H20 and Ascend 910B at about 128 GB/s. A horizon selected on one (device, batch, dispersion) triple is a different point on the curve for the others.

These three facts are the constraints on the model. Density determines how many distinct experts a step demands. Task identity determines which of those experts are worth a high-precision pin, on a slower timescale than a single layer. Bandwidth, batch, and dispersion move the horizon that hides transfers, and they do so only when absence is unavoidable. The rest of this section writes those constraints as a budget ratio and an overlap predicate.

2.3 The Budget Ratio

Let $P_{lo} = \sum_{\ell} E_{\ell} m_{\ell}^{lo}$ and $P_{hi} = \sum_{\ell} E_{\ell} m_{\ell}^{hi}$. Both sums are known once the packed representations have been materialized. The deployment ratio is $\Phi = B/P_{lo}$. Staging for a publish-before-reclaim overlap is a constant $M_{stage} = K \max_{\ell,b} m_{\ell}^b$, with K the maximum number of in-flight transitions. The controller spends $B - M_{stage}$. We fold M_{stage} into the definition of the usable budget from here on and continue to call it B , so the inequalities below are stated against the usable budget. Figure 1 is a schematic of the resulting regimes, not a measurement.

Phase I. If $B \geq P_{hi}$, every expert fits at high precision. The latency and quality costs of migration are both zero at the all-high assignment. DynaByte loads that assignment and disables both loops.

Phase II. If $P_{lo} \leq B < P_{hi}$, every expert fits at low precision and the all-high assignment does not fit. Absence is unnecessary. Any prefetch that moves an expert already resident adds a transfer and

does not remove one. The horizon that minimizes transfer latency is therefore $S^* = 0$. Replacing a low-precision resident with a high-precision resident costs $m_\ell^{\text{hi}} - m_\ell^{\text{lo}}$ additional bytes. When expert sizes are uniform within a layer and the same across layers, the maximum number of high-precision experts per layer satisfies

$$n_{\text{hi}} \leq \left\lfloor \frac{B - P_{\text{lo}}}{m^{\text{hi}} - m^{\text{lo}}} \cdot \frac{1}{L} \right\rfloor, \quad (1)$$

with L the number of MoE layers, the floor applying after a proportional split. The general case without that uniformity assumption is the integer knapsack that maximizes $\sum_\ell n_{\text{hi},\ell}$ subject to $\sum_\ell n_{\text{hi},\ell} (m_\ell^{\text{hi}} - m_\ell^{\text{lo}}) \leq B - P_{\text{lo}}$ and $0 \leq n_{\text{hi},\ell} \leq E_\ell$. We use the proportional split in the implementation and record the knapsack as an optimality gap to measure, not as the online algorithm. Identities inside the quota are the only online decision in this phase. They are ranked by the slow hotness score of Section 3.2.

As a numerical illustration of the algebra, not as a measured operating point: if $m^{\text{hi}} = 4 m^{\text{lo}}$ and $B = 1.5 P_{\text{lo}}$, then $\Phi = 1.5$ and the high-precision share of the pool is $(\Phi - 1)/(\rho - 1) = 1/6$. About one sixth of the experts can be pinned high, and the rest stay resident at low precision. A horizon controller that still evicts cold experts to “make room” destroys the floor that made $S^* = 0$ feasible.

Phase III. If $B < P_{\text{lo}}$, a complete low-precision floor does not fit. Some experts are absent, and a miss moves m bytes, where m is the size of the representation being fetched. Let $\hat{U}(S)$ be the set of experts the predictor expects to touch over the next S MoE layers, and let R be the set currently resident. The miss set is $\hat{U}(S) \setminus R$, and its transfer size is

$$M_{\text{miss}}(S) = \sum_{(\ell,e) \in \hat{U}(S) \setminus R} m_{\ell,e}, \quad (2)$$

where $m_{\ell,e}$ equals m_ℓ^{hi} if (ℓ, e) is a high-precision pin and m_ℓ^{lo} otherwise. Pins are not misses. A horizon S hides that transfer when two predicates hold simultaneously:

$$M_{\text{miss}}(S) \leq S \cdot T_\ell \cdot C, \quad (3)$$

$$\text{bytes}(R \cup \hat{U}(S)) \leq B. \quad (4)$$

Equation (3) is the overlap window. Equation (4) is cache pollution written as a hard constraint: a horizon whose speculative set does not fit is not a candidate, rather than a candidate with a penalty weight. Define

$$S^* = \min\{S \in [S_{\text{min}}, S_{\text{max}}] : (3) \text{ and } (4)\}, \quad (5)$$

and call the feasible set empty when no such S exists. The minimum is the latency-optimal choice under a lexicographic rule: meet the overlap constraint, then leave the largest possible slack $B - \text{bytes}(R \cup \hat{U}(S^*))$ for high-precision pins. A larger feasible horizon would also hide the mean transfer and would consume bytes that could have raised n_{hi} .

The empty-set case is the dense sub-phase. It is what Table 1 suggests during prefill at high batch: $\hat{U}(S)$ grows toward the full pool, $M_{\text{miss}}(S)$ exceeds both the overlap window and B , and increasing S makes (4) fail sooner. The latency-minimizing action is then $S^* = S_{\text{min}}$, with resident slots filled by the immediate layer’s demand and then by the slow hotness order. The non-empty case is the sparse sub-phase, typical of a small decode working set. There S^* is achievable, and every byte above $\text{bytes}(\hat{U}(S^*))$ is slack.

Tail demand uses the same predicate. Let $M_{\text{miss}}^{\text{p95}}(S)$ be (2) evaluated at the 95th percentile of the miss-set size observed over a sliding window of recent batches at this layer, with expert size m^{lo} for unpinned experts. The controller substitutes $M_{\text{miss}}^{\text{p95}}$ for M_{miss} in (3). The mean-demand horizon remains the quantity we plot against the model; the percentile horizon is what the runtime tracks.

Section 5 reports both, because a model that predicts the mean and misses the tail has not met a latency objective.

2.4 What the Model Predicts

The regime rules are predictions about latency, not about task score. Quality enters only as the use of slack. We separate them so that a failure of the hotness proxy does not get mistaken for a failure of the overlap identity.

- P1.** In Phase II the horizon that minimizes exposed expert-transfer latency is $S = 0$. A positive horizon increases migration bytes and does not reduce misses.
- P2.** In Phase III, when the feasible set is non-empty, S^* is non-increasing in C and non-decreasing in the byte size m of a prefetched expert. Replacing a low-precision prefetch with a high-precision prefetch shifts S^* up or empties the feasible set.
- P3.** At fixed $\Phi < 1$, a request can move from an empty feasible set during prefill to a non-empty set during decode. The minimizing action changes from “fill the immediate resident set, hold S at $S_{\min}$ ” to “track S^* , then pin from the slack.”
- P4.** Under the lexicographic rule, n_{hi} is bounded by the slack after S^* has been reserved. Raising S^* , raising m^{hi} , or entering the dense sub-phase all shrink that bound, including down to zero.

P1–P4 are the claims a joint re-measurement has to accept or reject. They also state the boundary policies precisely. A precision-only policy is the Phase II optimum, including its n_{hi} bound, applied at every Φ . When $\Phi < 1$ that policy has no feasible full floor; an implementation must either exceed B or silently fall back to a tighter quantization. A horizon-only policy is the Phase III rule with m fixed and n_{hi} fixed at zero, applied at every Φ . When $\Phi \geq 1$ it evicts experts the budget could have kept. A naive composition runs both controllers and lets each assume it may use B . The composition violates (4) whenever the pin quota and the speculative set are computed independently. Section 3 is the tracker that enforces one budget across both timescales. Section 5 measures the four predictions before it measures that tracker against the boundary policies.

2.5 Inputs the Model Does Not Observe

The overlap identity uses a scalar C . Concurrent compute traffic reduces the bandwidth that a side-stream copy actually receives. DynaByte estimates C from completed transfers during overlap, not from an idle bandwidth benchmark, and Section 5 reports the error of S^* under both estimators. Dequantization and the launch of a second grouped GEMM, one per live precision in the layer, add device time that is not a host-to-device transfer. Those times sit inside the measured T_ℓ . They shrink the overlap window rather than appearing as a separate miss term. The model does not predict task score. Hotness is a proxy for which expert should receive a slack byte, and it can be wrong while P1–P3 still hold. Kernel-launch overhead and proxy error are measured quantities in Section 5, not terms hidden inside Φ .

3 Design

DynaByte has two loops and one admission gate. The fast loop chooses S so that the resident set tracks (5). The slow loop chooses which experts occupy high-precision pins, and it changes the pin count only when the fast loop reports slack. The gate refuses any transition whose bytes are not already reserved. Figure 2 shows the path of one control period. The predictor that builds $\hat{U}(S)$ is deliberately local: it is the model’s pregate top- k when the checkpoint exposes pregate logits, and otherwise the empirical selection frequency of that layer over the last T_{fast} observations. A learned corrector is an ablation in Section 5, not part of the claim. If the predictor overestimates the miss

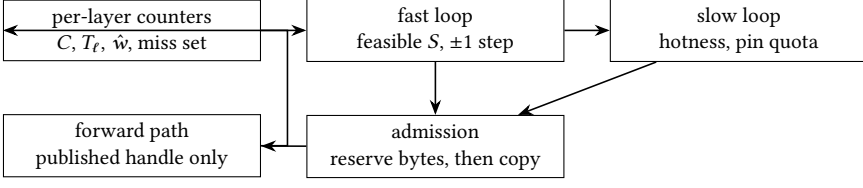


Fig. 2. One control period. The forward path reads a published handle and never waits on a migration. Counters from completed copies update C and T_l for the next period.

set, S^* grows until (4) fails and the dense rule takes over. Over-prediction therefore spends bytes on a wider resident set. It does not issue an unbounded prefetch.

3.1 Fast Loop

The fast loop runs once per control period of T_{fast} MoE layers. It reads the exponential moving averages of exposed wait and of the miss-set size, updates C from the bytes and duration of copies that completed on the migration stream, and updates T_l from CUDA events around the layer's expert kernels. It then evaluates (3) and (4) at the p95 miss size for every integer S in $[S_{\min}, S_{\max}]$. The range is the number of MoE layers in the model, so the scan is a few dozen predicate checks. The target is (5) when the feasible set is non-empty, and $S_{\min}$ when it is empty.

The applied horizon moves by at most one layer:

$$S \leftarrow S + \text{clip}(S^* - S, -1, 1), \quad (6)$$

except inside a hysteresis band. If the predicted change in exposed wait from taking the step is smaller than θ_h seconds, S stays put. Exposed wait for a candidate is $\max(0, M_{\text{miss}}^{\text{p95}}(S)/C - S \cdot T_l)$. The band is what keeps a noisy p95 estimate from flipping S every period. Because each step has size at most one, a constant target is reached in at most $S_{\max} - S_{\min}$ periods, and a target that jumps across the range cannot make the cache absorb that jump in one period. This is the same bound used to keep a horizon controller from oscillating, with the target now supplied by (5) instead of by a weighted difference of wait and eviction counts.

When the feasible set is empty the loop additionally marks the period dense. Prefetch admission then accepts experts from the current layer's predicted set first, and it does not extend the speculative set past $S_{\min}$. When the feasible set is non-empty the loop marks the period sparse and computes the slack

$$B_{\text{slack}} = B - \text{bytes}(R \cup \hat{U}(S)) \quad (7)$$

after the horizon step has been applied. B_{slack} is the only byte quantity the slow loop is allowed to turn into new high-precision pins.

3.2 Slow Loop

The slow loop runs every T_{slow} forward steps and is the only writer of high-precision identities. For each layer it maintains a score $s_{\ell,e}$. After a step with N_ℓ routed positions, the observation for expert e is the mean selected router mass

$$c_{\ell,e} = \frac{1}{N_\ell} \sum_{t=1}^{N_\ell} \mathbf{1}[e \in \mathcal{R}_{\ell,t}] p_{\ell,t,e}, \quad (8)$$

and $c_{\ell,e} = 0$ when e is not selected, so unused experts decay. The score is $s_{\ell,e} \leftarrow \alpha s_{\ell,e} + (1 - \alpha) c_{\ell,e}$. No task label and no online quality signal enter $s_{\ell,e}$.

In Phase II the pin count is the constant from (1). In Phase III the pin count starts at zero. On a sparse period the loop may increment $n_{hi,\ell}$ by one if $B_{slack} \geq m_{\ell}^{hi} - m_{\ell}^{lo}$ when a low-precision copy is resident, or $B_{slack} \geq m_{\ell}^{hi}$ when it is not. On a dense period the loop increments nothing. If the fast loop reports that the current pins have made (4) infeasible at every S , the slow loop decrements $n_{hi,\ell}$ by one. At most one increment or one decrement occurs per layer per slow period. That rate limit is what prevents a single prefill from demoting the decode pins and a single decode step from promoting a prefill-only expert.

Identities follow the count. An outsider replaces the lowest-scoring incumbent only when its score exceeds the incumbent by a margin δ . The margin is the mechanism that absorbs the near-ties visible when many experts have similar mass. A replacement is one demotion and one promotion, submitted as a pair, demotion first. Phase I performs neither step.

The separation of timescales is an assumption we measure rather than a parameter we hide. T_{slow} is set to a multiple of T_{fast} large enough that a pin change is constant across several horizon updates. Section 5 reports the waiting-time variance in the periods immediately after a pin change. If that variance dominates the variance from S itself, the lexicographic split is the wrong coupling and the paper's controller is the wrong design.

3.3 Admission and Pools

Device memory for expert weights is divided into per-layer pools of fixed-size blocks, one block size per precision. A high-precision block cannot be allocated from the low-precision pool. Fixed blocks remove variable-size external fragmentation from the expert allocator; the free path is a free list, not the general-purpose device allocator. A separate staging pool of K blocks, sized to the larger representation, covers the interval in which the new block is filled and the old block is still published.

Every promotion, demotion, and prefetch calls `try_reserve` before it acquires a block or enqueues a copy. The reservation checks three counters: the destination tier's capacity, the staging capacity, and the sum of published plus reserved expert bytes against B . A failed reservation leaves the published set unchanged. The caller retries on a later period from that published set, not from the set it wished to publish. Because reservations are subtracted from B_{slack} and from the fast loop's byte account before the copy starts, the pin loop and the prefetch loop cannot both spend the same byte.

Equation (4) is therefore an online invariant of DynaByte-owned memory: resident blocks, staging blocks, and the conversion workspace reserved at startup. It is not an invariant of the process. A KV-cache growth or a framework allocation beyond its declared reserve can still exhaust the device. Startup compares the sum of declared reserves against available memory and refuses to enter the loop if the sum exceeds the device. The implementation does not interpose on a later allocator call. The evaluation reports both the runtime's counters and the process peak so that a reserve violation is visible as a process-peak failure rather than as a counter failure.

3.4 Publication

A published handle is a precision tag, a device pointer, and the quantization metadata for one expert. The forward path resolves the router's expert identifiers to handles, takes a read lease, and records a per-stream last-use event after the expert kernel is launched. It never writes a handle in place.

A transition fills a destination block on the migration stream. The worker publishes the new handle only after that stream's completion event. The old handle is reclaimed only after its lease count is zero and every recorded last-use event has completed. The two waits cover the interval in which the forward path has read the old pointer and has not yet launched the kernel that uses it.

Demotion publishes the low-precision handle first and then reclaims the high-precision block under the same rule. The compute stream does not synchronize with the migration stream. A dispatch that arrives during the copy observes the old handle; the next dispatch observes the new one.

Tokens in a layer are issued in residency order. Tokens whose experts are already published run first. Tokens whose experts are in flight wait for that copy and run as the copy completes, concurrently with the resident tokens of the same layer. This overlap is the reason a single miss is charged as at most one transfer on the migration stream rather than as a stall in front of every expert kernel. The evaluation measures the exposed remainder of that transfer. The design bound is one transfer of size $m_{\ell,e}$; the bound is false if a miss triggers an eviction that reloads an expert the same step still needs, which is the failure mode (4) is there to forbid.

3.5 Control Period

Algorithm 1 is one period of the two loops. Phase classification uses the measured pool sizes and does not change unless B is reconfigured. The hotness update is omitted from the listing; it is (8) and runs on every forward step, while lines that change n_{hi} run only on slow periods.

3.6 What Is Intentionally Not Adapted

Top- k , expert architecture, and the router's parameters stay at the checkpoint values. DynaByte does not skip an expert to save a miss, because a skip changes the function being served. The low- and high-precision tensors are produced offline by the same quantization recipe for every expert; the online decision is which tensor is published, not a new set of scales. There is no per-request retuning of α , δ , θ_h , T_{fast} , or T_{slow} . Those constants are chosen on a calibration split and frozen. Section 5 states the split. A deployment that needs a different B changes Φ and therefore the phase, without a new calibration of the horizon.

4 Implementation

DynaByte is a Python runtime on PyTorch and HuggingFace Transformers [28, 38]. Expert modules are replaced by a dispatcher that resolves identifiers to published handles. Copies run on a dedicated CUDA stream. A CPU thread polls completion events, publishes handles, and returns reclaimed blocks to the free list. The fast and slow loops run on that thread. They do not launch GPU kernels other than the copies and the expert computations the forward path already performs. Control-period work is a scan of at most L horizon predicates plus an $O(E_\ell)$ score update per active layer, with L the number of MoE layers.

Both packed tiers are materialized in pinned host memory before the timed region. Materialization proceeds one layer at a time and releases the source weights of that layer once both tiers exist, so the host does not hold the unpacked checkpoint and both packed tiers together. A measured transition is a copy of an existing buffer. The run aborts if any expert fails to pack, register, or fit in its pool. There is no fallback onto unpacked weights.

The high-precision tier is FP16 and the low-precision tier is INT4 for models whose low-precision pool fits in the larger budgets we sweep. For Qwen3-Next-80B the low tier is INT2 and the high tier is INT4, because an INT4 pool of that model already occupies most of a 48 GB device and leaves no room for a second tier or for the KV reserve. The tier pair is a property of the checkpoint and is held fixed across the budget sweep. Changing B changes Φ , not the quantization recipe. Scales and zero-points travel in the handle metadata. A layer that contains both live tiers issues two grouped GEMM launches. The conversion workspace used by the INT4 and INT2 kernels is reserved in M_{stage} as a constant number of blocks per in-flight transition, matching the workspace the kernel actually allocates on first use. We record that size from a one-expert probe at startup rather than from a formula.

Algorithm 1 One DynaByte control period.

Require: usable budget B , bounds $[S_{\min}, S_{\max}]$, hysteresis θ_h , current horizon S

Require: resident set R , pin counts $n_{hi,\ell}$, measured C , T_ℓ , p95 miss bytes

```

1: if  $B \geq P_{hi}$  then
2:   load the all-high assignment and return ▷ Phase I
3: end if
4: if  $P_{lo} \leq B$  and  $B < P_{hi}$  then
5:    $S \leftarrow 0$  ▷ Phase II, Prediction P1
6:   set  $n_{hi}$  from the proportional form of Eq. (1)
7:   replace at most one pin per layer if the hotness margin  $\delta$  holds
8:   return
9: end if
10:  $F \leftarrow \{s \in [S_{\min}, S_{\max}] : \text{overlap and fit hold at p95 miss bytes}\}$ 
11: if  $F = \emptyset$  then
12:    $S^* \leftarrow S_{\min}$ ; dense  $\leftarrow$  true ▷ Prediction P3, dense
13: else
14:    $S^* \leftarrow \min F$ ; dense  $\leftarrow$  false
15: end if
16: step  $S$  toward  $S^*$  by at most one layer, unless the predicted wait change is below  $\theta_h$ 
17: if dense then
18:   admit prefetches from the current layer only; do not increment any  $n_{hi,\ell}$ 
19:   if pins make every horizon infeasible then
20:     decrement one  $n_{hi,\ell}$ 
21:   end if
22: else
23:   compute slack from Eq. (7)
24:   if slack covers one promotion then
25:     increment one  $n_{hi,\ell}$  and pin the highest-scoring unpinned expert
26:   end if
27: end if
28: submit admitted copies on the migration stream; publish on completion

```

C is the ratio of copied bytes to the duration of the migration-stream event, computed only for copies that overlapped expert compute. An idle H2D microbenchmark is recorded once per device and used solely as the alternative estimator in the model-error experiment. T_ℓ is the elapsed time of the expert kernels on the compute stream, which includes dequantization and both GEMM launches when both tiers are live. The p95 miss size uses a ring of the last 64 observations of $|\hat{U}(1) \setminus R|$ at that layer, scaled to horizon S by substituting the predicted union $\hat{U}(S)$. The ring is not a linear extrapolation of one layer's count, which would double-count experts that recur across layers.

Continuous batching is supported for the latency workload. The density $\hat{w}$ and the miss set are properties of the batch currently in the layer, so a mixed batch is controlled by its own union of experts rather than by any single request. Quality runs disable continuous batching and use greedy decoding so that a precision change cannot occur between the answer candidates of one multiple-choice item. All candidates of an item are scored in one unpadded batch.

Startup classification, pool capacities, and the frozen constants α , δ , θ_h , T_{fast} , and T_{slow} are written into the result artifact. The artifact also records the checkpoint revision, the quantization recipe,

the GPU UUID, the process peak sampled through NVML every 2 ms, and the runtime's own reservation counters. A point is reportable only when those counters stayed within B for the whole run or when the log shows a refused reservation. A run that exceeds B by allocating anyway is a bug, not a data point.

The prototype is single-GPU. Expert-parallel placements, in which a layer's experts are partitioned across devices, change both B and C per device and are out of scope. The accounting identity does not depend on that restriction; the implementation does.

Generative-model disclosure. SIGMETRICS 2027 requires the methods text to state the extent of generative-model assistance. [Authors: replace this paragraph before submission. State which prose was drafted with a language model, that the model, controller, and protocol are the authors' claims, and that every citation and measurement was checked by the authors. Do not leave this placeholder in a submission.]

5 Evaluation

The evaluation is a measurement of the model in Section 2.3 and of the tracker in Section 3. No end-to-end number from the two preceding studies is carried forward. Those studies used different budgets (a 48 GB residency target and a 20 GB cap), different model sets, and different primary metrics. Filling a table with either set would answer a different question than P1–P4. This section therefore specifies the protocol, the hypotheses, and the tables. Cells that require the new runs are marked unfilled.

5.1 Hypotheses

Each hypothesis names the prediction it tests and the comparison that rejects it.

H1 (P1). On every model–device pair with $\Phi \geq 1$, the exposed expert-transfer latency at $S = 0$ is no higher than at any $S > 0$ under the same pin quota. Rejection: some positive S reduces p95 exposed wait by a margin larger than the run-to-run standard deviation.

H2 (P2). On every pair with $\Phi < 1$ and a non-empty feasible set, the measured latency-minimizing horizon is non-increasing in C and non-decreasing in prefetch size m . Rejection: the minimizing S is unchanged, within one layer, when the prefetched representation switches between the two tiers or when the same binary is moved between the A6000 and H20.

H3 (P3). At a fixed budget with $\Phi < 1$, the feasible set is empty on the prefill of the batch-32 workload in Table 1 and non-empty on the following decode step, for at least one model. Rejection: the feasible-set predicate has the same truth value on those two steps for every model in the sweep.

H4 (P4). The largest n_{hi} that preserves a non-empty feasible set equals the slack bound from (7), to within one expert per layer. In the dense sub-phase that bound is zero. Rejection: adding a pin beyond the bound leaves p95 wait unchanged, or withholding a pin inside the bound reduces wait.

H5 (proxy). With the latency band held, hotness-ranked pins improve task score over a uniform low-precision assignment that uses the same B . Rejection: the task-score confidence intervals overlap on every model, or the pin policy meets the score gain only by leaving the latency band.

H6 (coupling). The naive composition, which computes a pin quota and a horizon independently against the full budget B , either exceeds B or has higher p95 exposed wait than DynaByte at the same budget. Rejection: the composition stays inside B and matches DynaByte within run-to-run noise on every phase.

H2 is the existence condition for a joint paper. If the minimizing horizon does not move when m changes, precision and prefetch are sequential optimizations and a single budget controller is unnecessary. The sweep reports that result even if it rejects H2.

5.2 Systems Under Test

All systems share the dispatcher, the pools, the NVML sampler, and the quantization recipe. They differ only in the policy that chooses the published tier and the horizon.

Uniform. Every expert is resident at the highest uniform tier whose pool fits in B . No online changes.

Static mixed. The Phase II quota is computed from B , identities are the top of a calibration ranking, and both the quota and the identities are frozen. This is the offline mixed-precision assignment [6, 12].

Precision-only. Phase II rules at every budget: full low-precision floor when it fits, hotness pins, $S = 0$. When the floor does not fit, the policy is infeasible and the run is recorded as infeasible rather than quietly quantized further.

Horizon-only. Phase III rules with $n_{hi} = 0$ and a single tier, the largest tier whose working set the horizon controller keeps inside B . This is the adaptive-horizon policy with fixed expert size [32, 34].

Miss-path mixed. Horizon-only, except a miss is fetched at low precision and promoted only after a second hit. This isolates the “cheapen the miss” policy [35] from a pin that is protected across periods.

Naive composition. Precision-only and horizon-only run together. Each computes its set against B . Admission is still on, so the second reservation fails instead of overflowing; we count failed reservations and the resulting misses.

DynaByte. Algorithm 1.

Oracle horizon. For each workload, an offline sweep of S and n_{hi} selects the pair with the lowest p95 exposed wait subject to bytes $\leq B$. The oracle is a ceiling for the latency model, not a deployable policy. It is computed on the calibration prompts and also, separately, on the test prompts. The gap between those two oracles is the price of non-stationarity.

We do not report a comparison against a system we could not execute on the same checkpoint. Where a published runtime does not load one of the checkpoints, that cell is absent, not approximated.

5.3 Testbed, Models, and Workloads

Devices are an NVIDIA RTX A6000 (48 GB), an NVIDIA H20, and an Ascend 910B, the three machines on which the preliminary horizon sweep was taken. The A6000 is the only device on which the full quality suite is required. H20 and 910B are required for H2, because they change C . If the 910B software stack cannot execute the INT4 kernels for a checkpoint, that pair is dropped and the drop is reported; it is not replaced with a different quantization.

Models are Qwen3-30B-A3B, Qwen3-Next-80B-A3B, and Phi-3.5-MoE, plus DeepSeek-V2-Lite, Qwen1.5-MoE, and Qwen2-MoE so that the 20 GB region of the sweep contains models whose routing we have already traced [1, 3, 29, 36, 41]. Table 2 records the structural parameters. The budget sweep is $B \in \{12, 16, 20, 24, 32, 40, 48\}$ GB on the A6000, clipped to the device capacity on the others, and further clipped to P_{hi} because budgets above Phase I are idle. For each pair we report Φ from the measured P_{lo} , not from the nominal parameter count.

Latency requests are drawn from ShareGPT [2], bucketed by prompt length, with at most 50 requests per bucket and length variation held within 5% inside a bucket. This is a public conversational corpus; the runs send no data to an external service. Each latency configuration uses

Table 2. Model structure. Activated parameters are the vendor figures. Φ is filled from measured P_{lo} at each B , not from this table.

Model	Params	Active	Layers	Experts	Top- k
Qwen3-30B-A3B	30.5B	3.3B	48	128	8
Qwen3-Next-80B-A3B	80B	3B	48	512	10
Phi-3.5-MoE	42B	6.6B	32	16	2
DeepSeek-V2-Lite	[fill]	[fill]	[fill]	[fill]	[fill]
Qwen1.5-MoE	[fill]	[fill]	[fill]	[fill]	[fill]
Qwen2-MoE	[fill]	[fill]	[fill]	[fill]	[fill]

five warmup iterations and 100 measured iterations, repeated for five seeds. We report the mean, the median, and the 95th and 99th percentiles of exposed wait, of miss-path time, of time to first token, and of time per output token. Percentiles are computed on the pooled raw samples of a seed and then summarized across seeds by the median and the range. Exposed wait is the time a layer is stalled on a missing expert after subtracting the overlap with that layer’s resident expert compute. Miss-path time is the migration-stream duration and is allowed to be larger than exposed wait.

Quality uses WikiText-2 perplexity [26] and the task scores of MMLU-Pro, GPQA, GSM8K, and HumanEval [5, 8, 30, 37]. The item sets are the pinned subsets already used in our routing traces: 200 MMLU-Pro items, the GPQA-Diamond set, 200 GSM8K items, and the full HumanEval set. Perplexity uses at most 128 non-overlapping windows of 2,048 tokens. Multiple-choice items are scored by the conditional log-likelihood of the full answer label. HumanEval is pass@1 under greedy decoding after executing the official tests. Accuracy intervals are Wilson intervals at 95%. We do not pool tasks into a single significance test. A model is a separate experiment.

Calibration and test are disjoint. Hotness initialization, the frozen controller constants, and the oracle-horizon selection use a calibration slice of ShareGPT and a calibration slice of WikiText. Test prompts, including the three task workloads used for the hot-set disjointness check, are held out. Constants are not refit per device. C and T_l are measured online and are not calibrated constants.

5.4 Experiments

E0: repeat the phenomena on one stack. Before any policy comparison we remeasure Table 1, the Layer-15 top-10 overlap across WikiText, GSM8K, and HumanEval, and the latency-versus- S curve at $B = 20$ GB. E0 uses the uniform low tier and a fixed horizon, with DynaByte’s loops off. If the unique-expert ratio does not move by at least 30 percentage points from prefill to decode on Qwen3-30B at batch 32, or if the latency curve is not U-shaped on at least two models, the preliminary motivation does not survive the unified runtime and the later experiments are not informative. E0 is reported in full, including a negative outcome.

E1: the phase diagram. For each model on the A6000 we plot p95 exposed wait and WikiText perplexity against B , with one curve per system in Section 5.2. The horizontal axis is annotated with the measured $\Phi = 1$ and $\Phi = P_{hi}/P_{lo}$ points. H1 requires the precision-only curve and the DynaByte curve to meet for $\Phi \geq 1$, and the horizon-only curve to lie above them by the migration it still performs. H6 is read from the same figure: the naive composition either has no point, because admission refused the dual reservation, or lies above DynaByte. Table 3 is the numerical companion, filled per regime rather than by a single speedup.

E2: does S^ move with m and with C ?* On each Phase III configuration we sweep S with prefetch size fixed at m^{lo} and again at m^{hi} , pins disabled, and we record the minimizing S . We repeat the

Table 3. Phase diagram on the A6000, p95 exposed wait (ms) and perplexity. One row group per model. Unfilled cells are the joint re-measurement.

Model	Regime	Uniform	Prec.-only	Horiz.-only	DynaByte
[model]	$\Phi \geq \rho$	[ms / ppl]	[ms / ppl]	[ms / ppl]	[ms / ppl]
[model]	$1 \leq \Phi < \rho$	[ms / ppl]	[ms / ppl]	[ms / ppl]	[ms / ppl]
[model]	$\Phi < 1$, sparse	[ms / ppl]	[infeasible or ms]	[ms / ppl]	[ms / ppl]
[model]	$\Phi < 1$, dense	[ms / ppl]	[infeasible or ms]	[ms / ppl]	[ms / ppl]

Table 4. Task score at the latency band of Section 5.4, A6000. Wilson 95% intervals travel with each accuracy cell in the artifact.

Model	Task	Uniform	Static mixed	Prec.-only	DynaByte
[model]	MMLU-Pro	[score]	[score]	[score]	[score]
[model]	GPQA	[score]	[score]	[score]	[score]
[model]	GSM8K	[score]	[score]	[score]	[score]
[model]	HumanEval	[score]	[score]	[score]	[score]
[model]	WikiText ppl	[ppl]	[ppl]	[ppl]	[ppl]

low-precision sweep on H20 and on the 910B. H2 is a statement about the argmin, not about the height of the curve. The model error is $|S_{\text{pred}}^* - S_{\text{meas}}^*|$, with S_{pred}^* computed from (5) twice: once with idle C , once with in-overlap C . We report the mean absolute error in layers. A useful model is one whose in-overlap error is at most one layer on a majority of configurations. An idle- C error that is large while the in-overlap error is small is evidence for the contention caveat in Section 2.5, not evidence against the overlap identity.

E3: inside one request. For the batch-32 ShareGPT buckets we log, per request, the feasible-set predicate on the last prefill layer and on each of the first 32 decode steps, together with the applied S and n_{hi} . H3 is a predicate on that log. We also report the number of periods the applied S takes to come within one layer of a new S^* after the prefill-to-decode boundary, and the migration bytes moved during those periods. The hysteresis and the ± 1 step are acceptable only if that migration stays below the migration of an ablation that sets S to S^* in one period.

E4: slack and pins. At three budgets in Phase III (sparse) we compare $n_{\text{hi}} \in \{0, n^*, n^*+1\}$, where n^* is the slack bound. H4 predicts that n^*+1 either fails admission or increases p95 wait, and that n^* does not. The dense rows of the same table use the batch-32 prefill. H4 predicts $n^* = 0$ there.

E5: quality at a fixed latency band. On the A6000 we take the largest B in Phase II and the largest B in Phase III at which DynaByte’s p95 time-to-first-token stays within 10% of the uniform-low baseline’s. If no Phase III budget meets that band, the Phase III quality comparison is reported as “band infeasible” and H5 is not claimed there. Inside the band we report task scores for uniform, static mixed, precision-only, and DynaByte. Table 4 has one block per model. The static-mixed column is the comparison that tests whether online identity changes matter once the quota is held fixed. Intervals that overlap are reported as overlaps.

E6: invariant, overhead, and ablations. For every reported run we record the maximum of the runtime byte counter and the NVML process peak. A counter above B is a failed implementation. A process peak above the device, while the counter stays inside B , is a reserve violation and is reported separately. Controller CPU time per fast period and per slow period is reported as a distribution.

The design target is that the fast period stays well below T_ℓ ; the evaluation states the measured fraction of T_ℓ rather than a microsecond budget chosen in advance.

Ablations remove one mechanism at a time on the A6000, on one Phase II model and one Phase III model: no hysteresis on pins, no ± 1 limit, idle C instead of in-overlap C , mean miss size instead of p95, no cache-aware issue order, and a single pool instead of per-tier pools. A further ablation replaces the pregate-or-frequency predictor with a CPU random forest trained on calibration tuples of token identity, layer, and horizon. That ablation is informative only when E2 shows a residual wait that the overlap model attributes to miss-set error rather than to C or m . If the residual is small, the forest is not part of the system we recommend.

5.5 What We Will Not Claim from This Protocol

A single speedup against uniform quantization is not a result under this protocol. Uniform quantization is the Phase I optimum and a feasible Phase II baseline; it is infeasible in part of Phase III, and a speedup there would compare against a system that does not fit. A task-score gain with an unstated latency regression is not evidence for H5. A horizon policy that wins at one batch size and one device is the behavior P2 predicts for a fixed S , and it is not a competing system result unless the policy is the horizon-only controller, which already adapts S .

6 Discussion

6.1 What Would Falsify the Account

The account is a latency model plus a proxy that spends slack. They fail separately. If E2 finds that the minimizing horizon is insensitive to prefetch size, then (3) is missing a term that dominates m , and precision does not belong in the horizon calculation. The likely missing term is a kernel-launch or dequantization cost that scales with the number of experts rather than with bytes. In that world the right design is the precision-only policy whenever the floor fits, and a horizon policy whose S is chosen from T_ℓ alone whenever it does not. DynaByte's coupling would be an unnecessary constraint.

If E2 finds that idle C and in-overlap C produce horizon errors of several layers, the scalar overlap identity is the wrong plant model even if a closed-loop controller can still hunt for a good S . The paper would then be a control paper about tracking a noisy wait signal, which is a weaker and different claim than P2. We would rather report that negative than retune θ_h until the controller looks accurate.

If E5 finds that hotness-ranked pins do not separate from static mixed precision inside the latency band, the slow loop's identity updates are not earning their migration traffic. The quota rule in (1) and the slack rule in (7) could still be right. The system we would recommend is then a static ranking inside a controller-chosen quota, and the task-disjoint top-10 result in Section 2.2 would be a statement about calibration shift that our online score failed to track. That outcome is reportable from the same tables.

6.2 Limitations

The implementation is one GPU. An expert-parallel deployment splits E_ℓ across devices, and Φ becomes a per-device quantity that also depends on the router's balance. Nothing in the model forbids that reading, and nothing in the prototype measures it.

Host memory holds both packed tiers for the life of the process. The device budget is the object of the invariant; the host duplicate is a cost paid at load time. A deployment that cannot store both tiers must quantize on the promotion path, which adds a term to transition latency that Algorithm 1 does not hide.

The lexicographic rule prefers a feasible horizon over a pin. Under a very tight B the slack is zero for the entire request and DynaByte reduces to the horizon-only policy. That reduction is the correct output of P4, and it means the system has no quality advantage over horizon-only in the tightest Phase III budgets. We state the quality claim only inside the band defined in E5.

Shared experts that the checkpoint always executes are charged to the non-expert reserve, not to B . Qwen3-Next has one such expert per layer. Treating it as a routed expert would inflate P_{10} and shift Φ .

The p95 ring is a short history. A workload shift shorter than the ring, or a slow loop that has not yet decayed the previous task's scores, mis-states both S^* and the pin identities. E3's text-to-code boundary, using the three workloads whose top-10 sets were disjoint, is the measurement of that lag. We do not claim a bound on lag in periods; we report it.

Cache-aware issue order assumes that tokens in a layer may execute in any order relative to each other. That is true for the standard grouped-GEMM dispatch and false if a future kernel fused the experts in identifier order. The ablation that disables reordering measures the exposed-wait gap on today's kernels.

6.3 Threats to Validity

The preliminary numbers in Section 2.2 and the results of E0 are different samples until E0 is run. We will replace Table 1 with the E0 table if the two disagree, rather than explaining the disagreement in a caption.

ShareGPT buckets under-represent very long contexts. A 128k-token prefill changes both the unique-expert ratio and the KV reserve, and the KV reserve eats into the device before B does. Our budgets assume the declared KV reserve is honored. They do not describe a serving stack that over-admits context length.

Quality subsets are a few hundred items. Wilson intervals on those subsets are wide for small effects. H5 is a claim about separation of intervals, so a small true effect on this item set will correctly fail to reject the null. We will not respond to that failure by switching to a larger, post-hoc item set.

Ascend and NVIDIA stacks do not share a kernel implementation. A C comparison across them confounds bandwidth with software. H2's device clause is therefore stated as a change in the measured in-overlap C , and we report that C , rather than as a pure hardware effect.

6.4 Reproducibility and Ethics

The latency corpus is a public conversational dataset. Quality benchmarks are public exam-style and code datasets. The study does not recruit participants, does not collect new user data, and does not transmit prompts to a third-party API. Artifacts will contain prompt identifiers and scores, not a redistribution of any dataset whose license forbids it.

The release accompanying an accepted paper is the controller, the pool runtime, the trace utilities that compute the unique-expert ratio and Φ , and one configuration file per reported cell. Random seeds, checkpoint revisions, and the calibration-test split identifier are part of each artifact. We will not release a binary that omits the reservation counters. A result that cannot be checked against B is not a result under this protocol.

7 Related Work

We organize prior work by which decision it holds fixed. The relevant decisions are the three residency states and the horizon. A system that optimizes one of them under a fixed setting of the others is a boundary policy of Section 2.3, and it is a baseline in Section 5.2 when we can run it.

7.1 Prefetch Horizons and Expert Caches

Cross-layer prefetch overlaps the transfer of experts predicted for the next S layers with compute [13, 18, 34]. ProMoE and the pregate predictor of Eliseev and Mazur select S as a deployment parameter. The preliminary sweep in Section 2.2 is a measurement of that parameter's U-shaped latency on one device family. Expert caches choose which resident expert to evict and which missing expert to stage [4, 21, 24, 40, 43, 46]. MoE-Infinity builds a locality-aware cache; MoE-Lightning rebalances residency across a memory hierarchy during a run. These systems are Phase III designs with a single byte size per expert. Their horizon, when they have one, is not required to be the minimum feasible S under (3) and (4), and a high-precision pin is not a protected class of resident.

ExpertFlow appears as two distinct lines. He et al. allocate tokens to experts to improve activation efficiency [16]. Shen et al. adapt expert scheduling and coordinate expert memory under offload [32]. Both change which experts move and when. Neither states a budget ratio that disables the horizon once a low-precision floor fits, which is the Phase II prediction. We cite them as scheduling policies, and the horizon-only baseline is our implementation of the adaptive-horizon rule rather than a claim that we executed either codebase unchanged.

Placement systems that colocate frequently co-activated experts, or that avoid cross-device transfers by topology-aware routing, reduce communication for multi-device inference [9, 42]. They assume the experts they place can stay resident at the precision the checkpoint was loaded in. That is Phase I. DynaByte does not change placement across devices.

7.2 Quantization

GPTQ, AWQ, SmoothQuant, and LLM.int8() produce a static low-bit representation [11, 15, 25, 39]. MoE-specific recipes assign bit widths from expert sensitivity, activation frequency, or router statistics, still offline [6, 7, 12, 20, 44]. The static-mixed baseline is this family: a quota and an identity ranking frozen after calibration. The preliminary hot-set measurement is why a frozen ranking is a different policy from the slow loop. MxMoE additionally fuses mixed-precision expert GEMMs [12]. We treat that kernel as complementary. Our overhead account charges two launches when both tiers are live, and a fused kernel would change T_ℓ , which the fast loop already measures rather than assumes.

7.3 Precision on the Transfer Path

HOBBIT combines a multi-level expert cache with mixed precision so that a miss copies fewer bytes [35]. DyMoE classifies expert importance online, applies a depth-aware 8/4/0-bit policy, and prefetches missing versions from host memory or SSD for edge-class budgets of 12–24 GB [17]. A zero-bit assignment drops the expert and changes the served function. DynaByte does not drop experts. The substantive difference, including against HOBBIT's cheaper miss path, is what a byte is allowed to buy. In those systems a low-precision copy is a transfer optimization for an expert that missed. In DynaByte a low-precision copy is a resident state that can remove the miss entirely when $\Phi \geq 1$, and a high-precision copy is a pin paid for from slack after S^* is feasible. The miss-path mixed baseline exists so that this difference is a measured gap rather than a taxonomic one. DyMoE's edge budgets sit in Phase III for the models in Table 2; the sweep includes that region instead of arguing from the device class.

7.4 Serving Engines

vLLM, SGLang, and FlexGen manage KV-cache pages, request scheduling, or offload of a dense model [22, 33, 45]. Their allocators are the reason DynaByte's invariant is stated on expert pools and not on the process. A paged KV allocator that stays inside its reserve is compatible with the

startup check. An allocator that grows the KV cache from the same device heap as expert blocks is the reserve violation E6 is instrumented to catch. We do not modify those engines in this paper. The dispatcher is a Transformers module so that the policy comparison is not confounded with an engine's scheduler.

8 Conclusion

Expert-weight memory on one accelerator is one byte budget. $\Phi = B/P_{l_0}$ decides whether that budget can hold a low-precision copy of every expert. When it can, the horizon that minimizes exposed transfer latency is zero, and the leftover bytes are a high-precision pin quota whose identities follow the router on a slow loop. When it cannot, the horizon is the smallest value that hides the predicted miss set inside both the overlap window and the budget, and pins are funded only from the slack above that horizon. Prefill and decode move a request between those two sub-cases without changing Φ .

DynaByte classifies the regime from measured sizes, tracks the predicted horizon by one layer per period, and admits a precision change only against reserved bytes. Precision-only residency, horizon-only prefetch, and a naive composition of the two are the boundary policies of the same model. The measurement protocol tests the overlap predictions first, including the prediction that the minimizing horizon moves with prefetch size. If that prediction fails, the coupling is the wrong design and the tables will show it. Until those runs exist, this paper is a model, a controller, and a protocol, not a performance result.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219 [cs.CL]
- [2] Anonymous Contributors. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Hugging Face dataset.
- [3] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).
- [4] Shiyi Cao et al. 2024. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. *arXiv preprint arXiv:2411.11217* (2024).
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]
- [6] Krishna Teja Chitty-Venkata, Jie Ye, and Murali Emani. 2025. MoPEQ: Mixture of Mixed Precision Quantized Experts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4023–4032.
- [7] Mohammed Nowaz Rabbani Chowdhury, Kaoutar El Maghraoui, Hsin-yu Tsai, Naigang Wang, Geoffrey W. Burr, Liu Liu, and Meng Wang. 2026. Efficient Quantization of Mixture-of-Experts with Theoretical Generalization Guarantees. In *The Fourteenth International Conference on Learning Representations*.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [10] DeepSeek-AI. 2024. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [11] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, Vol. 35. 30318–30332.
- [12] Haojie Duanmu, Xiuhong Li, Zhihang Yuan, Size Zheng, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2025. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. *arXiv preprint arXiv:2505.05799* [cs.LG]
- [13] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).

- [14] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [15] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [16] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2024. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. *arXiv preprint arXiv:2410.17954* (2024).
- [17] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. 2026. DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge. *arXiv preprint arXiv:2603.19172*. [arXiv:2603.19172](https://arxiv.org/abs/2603.19172) [cs.LG]
- [18] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
- [19] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [20] Young Jin Kim, Raffy Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv preprint arXiv:2310.02410*. [arXiv:2310.02410](https://arxiv.org/abs/2310.02410) [cs.LG]
- [21] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6710–6720.
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [23] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.
- [24] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive gating in mixture-of-experts based language models. *arXiv preprint arXiv:2310.07188* (2023).
- [25] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of machine learning and systems* 6 (2024), 87–100.
- [26] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. [arXiv:1609.07843](https://arxiv.org/abs/1609.07843) [cs.CL]
- [27] NVIDIA Corporation. 2024. NVIDIA RTX A6000 Datasheet. <https://resources.nvidia.com/en-us-briefcase-for-datashets/proviz-print-nvidia-1?ncid=no-ncid>. Accessed: 2025-06-14.
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [29] Qwen Team. 2025. Qwen3-Next-80B-A3B-Instruct Model Card. <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct>. Accessed: 2026-07-29.
- [30] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling*.
- [31] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.
- [32] Zixu Shen, Xexin Chu, Yifan Zhang, Dawei Xiang, Runxin Wu, and Wei Zhang. 2025. ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference. *arXiv preprint arXiv:2510.26730*. [arXiv:2510.26730](https://arxiv.org/abs/2510.26730) [cs.PF]
- [33] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. *Proceedings of Machine Learning and Systems* 5 (2023), 1–16.
- [34] Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134* (2024).
- [35] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433* (2024).

- [36] Qwen Team. 2025. Qwen3 Technical Report. [arXiv:2505.09388](https://arxiv.org/abs/2505.09388) [cs.CL]
- [37] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhramil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. 2024. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems* 37 (2024), 95266–95290.
- [38] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [39] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*. PMLR, 38087–38099.
- [40] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361* (2024).
- [41] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [42] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K DK Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 915–925.
- [43] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. EdgeMoE: Fast on-device inference of MoE-based large language models. *arXiv preprint arXiv:2308.14352* (2023).
- [44] Jinhao Zhang, Yunquan Zhang, Boyang Zhang, Zeyu Liu, and Daning Cheng. 2025. MoQE: Improve Quantization Model performance via Mixture of Quantization Experts. *arXiv preprint arXiv:2508.09204* (2025).
- [45] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Liang Shi, Chenguang Wu, et al. 2024. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104* (2024).
- [46] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.